Taiwanese Bankruptcy Prediction

대만 기업 파산 예측 데이터톤

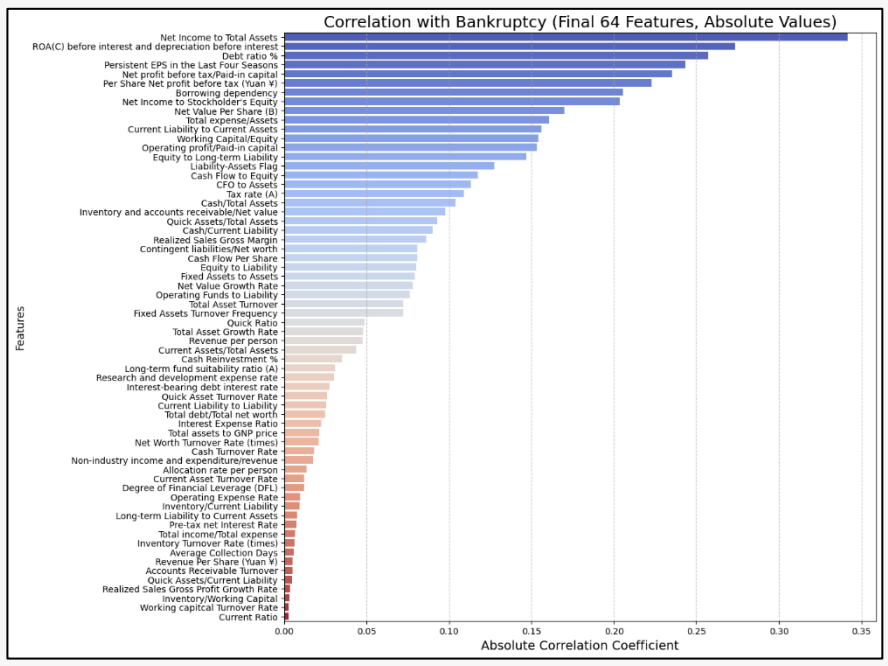
EPOCH 병정개미

EDA (데이터 이해)

	feature	VIF
77	Current Liability to Liability	172710524136.00613403
64	Current Liabilities/Liability	73459794597.19927979
56	Current Assets/Total Assets	44841161427.30332947
54	Working Capital to Total Assets	14234352364.31833458
38	Net worth/Assets	7463463854.53319931
60	Current Liability to Assets	4238887463.59987879
66	Current Liabilities/Equity	351794730.72367042
78	Current Liability to Equity	316448065.32637143
37	Debt ratio %	230320699.06554544
6	Operating Profit Rate	184001882.64757615
7	Pre-tax net Interest Rate	67842901.54109038
4	Operating Gross Margin	47145380.54761188
89	Gross Profit to Sales	40181000.69623962
9	Non-industry income and expenditure/revenue	25834950.90692336

FEATURE ENGINEERING

- 다중공선성 제거
- 상환 능력 지수 파생변수 생성
- 비효율적 좀비 기업 지표 파생변수 생성
- 자본 구조의 취약성 지표 파생변수 생성
- 타겟 변수와 상관관계 상위 40개 셀렉



모델 설계

- 사용 모델 (ex. LightGBM, XGBoost 등)
- 선택 이유 (간단히)
- baseline → 개선 흐름

"왜 이 모델인지" 설명

- 모든 test를 타겟 1로 해서 recall1로 만들고, precision확인 -> 총 몇 개의 1이 존재하는지 파악 -> 66개
- 트리 모델 -> 25개의 변수가 0과 1사이를 벗어났기에
- 우리의 전략은 66개를 전부 맞추는 즉, recal을 1로 만드는 전략
- 이를 위해 안정성을 올림 -> 최대한 잘 맞추도록 진행
- 앙상블 썼다. 주로 RF + XGBoost 섞기

파라미터 튜닝

- 주요 하이퍼파라미터
- 튜닝 방법 (grid / random / optuna 등)
- 성능 변화 (간단 비교)

"깊이보다는 “논리성”

성능 평가

- Validation 방식 (K-fold 등)
- CV score
- Leaderboard score
- (가능하면) train vs val 비교

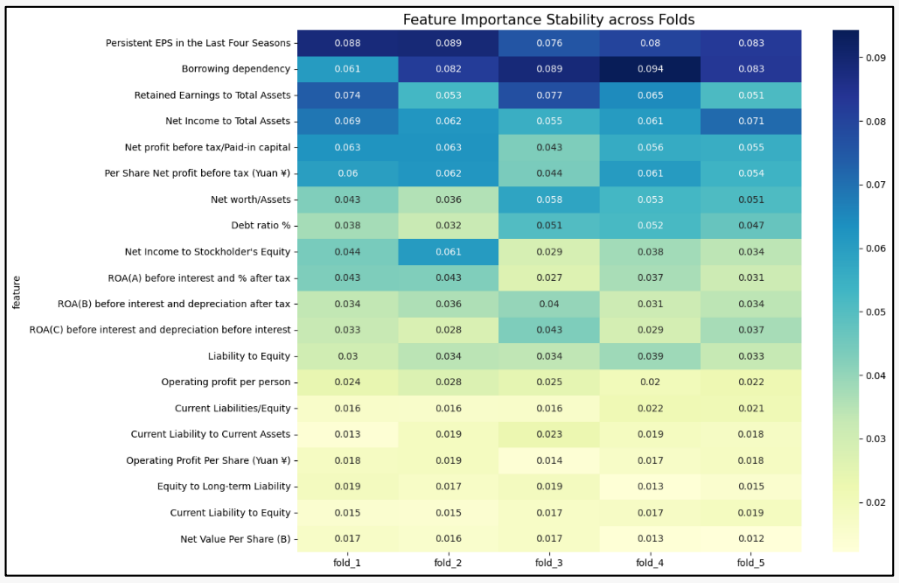
“이 모델이 진짜 잘 된 건지” 증명

- 5 – fold CV를 썼음.

모델 해석

주요 3개의 feature에 대한 해석

- **Persistent EPS(지속적 주당 순이익)**
  다음 1년동안에도 지난 1년 간 번 돈을 계속 벌 수 있는지 나타내는 지표로 기업의 안정성을 나타냄
- **Borrowing Dependency (차입 의존도)**
  기업이 부채 의존도로 클수록 기업의 파산 위험성 시사
- **Retained Earnings / Total Assets (이익잉여금/총자산)**
  총자산 대비 이익의 비중으로, 클수록 기업의 성장성을 의미



결론 및 인사이트

Random Forest를 기반으로 모델을 설계하여 이상치가 많은 불균형 데이터인 파산 데이터를 다루고자 함.

XGBOOST, MLP Classifier와 Ensemble하여 모델의 성능을 올려보고자 하였으나 f2 score에는 큰 변화가 없어 Random Forest만을 단독적으로 사용한 모델을 최종 모델로 결정.

또한 불균형 데이터를 다루기 위해 samplin을 사용하기도 하였지만, 역시 성능에는 큰 차이가 없어 모든 데이터를 사용하기로 결정.

VIF를 기반으로 각 feature들의 중요도를 확인하여 재무제표의 어떤 변수들이 기업 파산 예측에 있어 큰 영향을 주는지 확인할 수 있었음.

- 이상치 처리에 대한 방향성 제시
  해당 모델에서는 feature selection을 제외하고는 모든 데이터를 사용하였기에 이상치 처리가 전혀 이루어지지 않았음.
하지만 모델의 적합에 있어 이상치 처리는 매우 중요한 문제이기에 다양한 측면에서 고려하였음.

1. 이상치 처리를 따로 하지 않고 전체 데이터 대입 -> Tree 기반 모델 설계이기에 이상치의 영향을 덜 받음
2. 큰 이상치를 보인 데이터 (1 이상 값을 가진 데이터)를 새로운 추가 변수로 설정 -> 10만 이상의 값을 가지는 outlier에 대해 새로운 변수로 설정하고, scaling하는 과정을 거쳐 모델링해보았으나 눈에 띄는 성능 변화는 없었음.
3 .